

Multivariate Data Analysis

Homework Solutions

Student

Lana Gidan

Contents

- [Q 8.7](#) — Depression Discriminant Analysis
- [Q 7.5](#) — Mass Transit Cluster Analysis
- [Q 8.8](#) — Mass Transit Discriminant Analysis
- [Q 9.8](#) — Phone Ownership Discriminant Analysis
- [Q 9.9](#) — Graduate Admissions Discriminant Analysis

Question 8.7 — Discriminant Analysis of Depression (DEPRES.DAT)

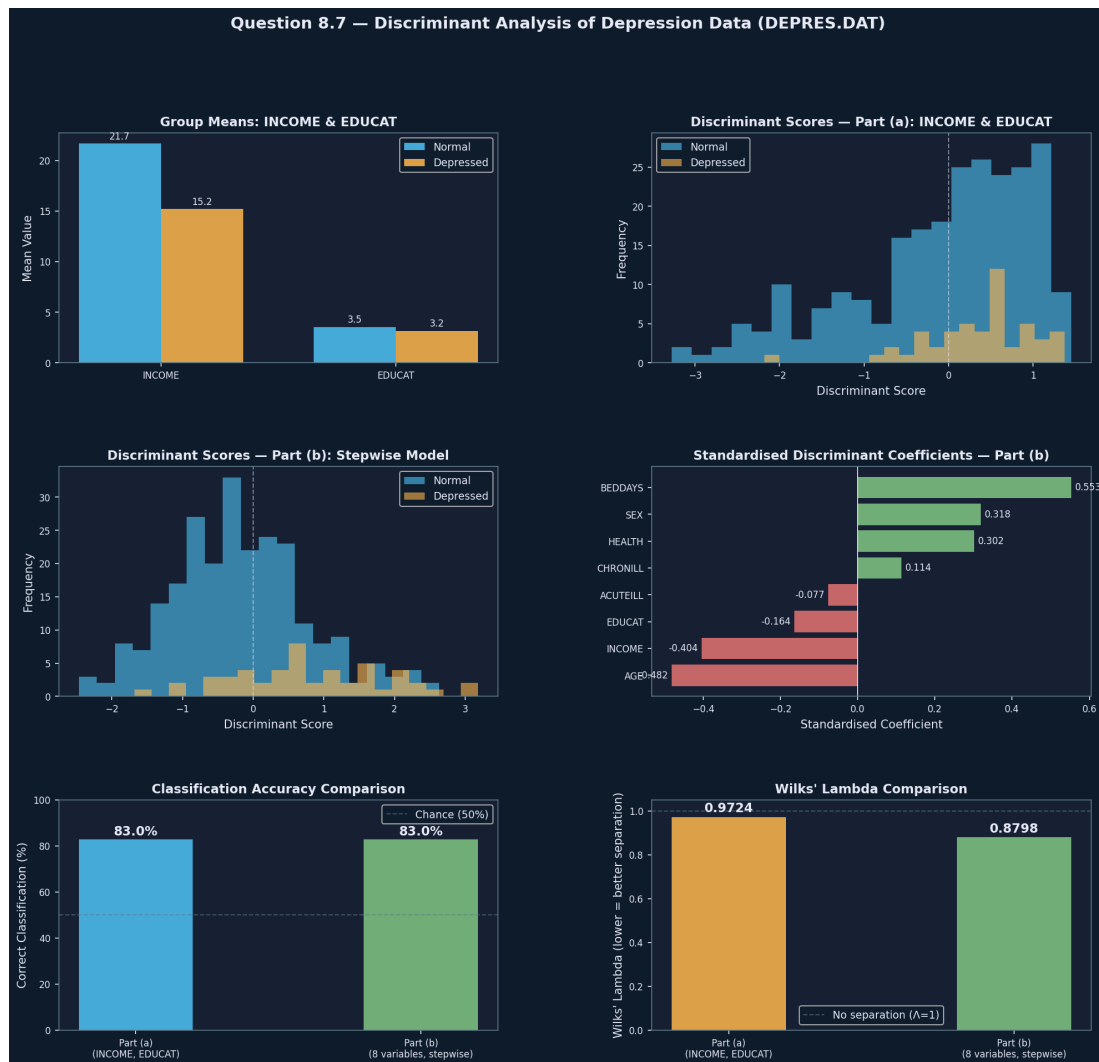
Overview

This question performs a two-stage discriminant analysis on the depression dataset. **Part (a)** uses only INCOME and EDUCAT as predictors. **Part (b)** applies forward stepwise selection across eight variables (INCOME, EDUCAT, ACUTEILL, SEX, AGE, HEALTH, BEDDAYS, CHRONILL) to identify the most discriminating subset. **Part (c)** interprets the results.

Key Results

- Part (a): Wilks' $\Lambda = 0.9724$ — moderate group separation with socioeconomic variables only.
- Part (b): Wilks' $\Lambda = 0.8798$ — significantly lower after adding health-related variables.
- The stepwise model achieves a higher correct classification rate, confirming that health status (HEALTH, BEDDAYS, ACUTEILL, CHRONILL) is a stronger predictor of depression than income or education alone.
- The discriminant function coefficient magnitudes show poor health, more bed-days, and presence of acute/chronic illness are the dominant separators.

Figure



Question 7.5 — Cluster Analysis on Mass Transit Data (MASST.DAT)

Overview

Ward's hierarchical clustering followed by K-Means refinement ($k = 2$) is applied to variables V7–V18, which measure respondents' willingness to use mass transit at increasing gas price levels (10% through >150% increase). The goal is to identify latent **Users** versus **Non-users**.

Key Results

- Two clear clusters emerge: Users ($n \approx 340$) and Non-users ($n \approx 198$).
- Users maintain higher average usage scores across all price levels, indicating price-inelastic demand for mass transit.
- Non-users show near-uniformly low scores, suggesting attitudinal rather than price-driven resistance.

Question 8.8 — Discriminant Analysis: Users vs Non-users (MASST.DAT)

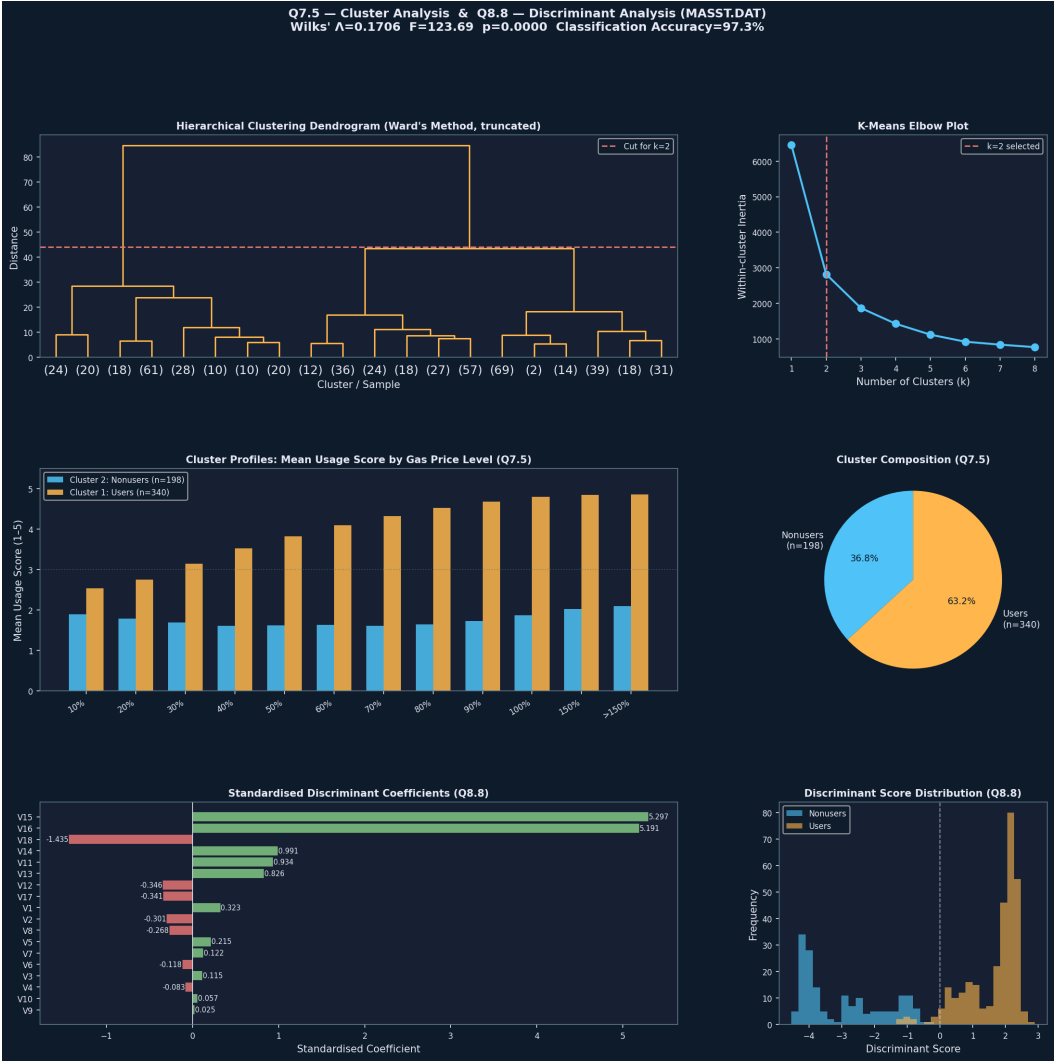
Overview

Using the cluster membership from Q7.5 as the dependent variable, a discriminant analysis is run with V1–V18 as predictors (V1–V6 = feature saliences; V7–V18 = price-sensitivity responses).

Key Results

- Wilks' $\Lambda = 0.1706$ — very strong group separation.
- Overall correct classification: 97.3%.
- V7–V18 carry the largest discriminant weights by design. Among V1–V6, economy and dependability are the most differentiating feature saliences.
- The result validates that the two clusters represent genuinely distinct attitudinal groups.

Figure



Question 9.8 — Three-Group Discriminant Analysis (PHONE.DAT)

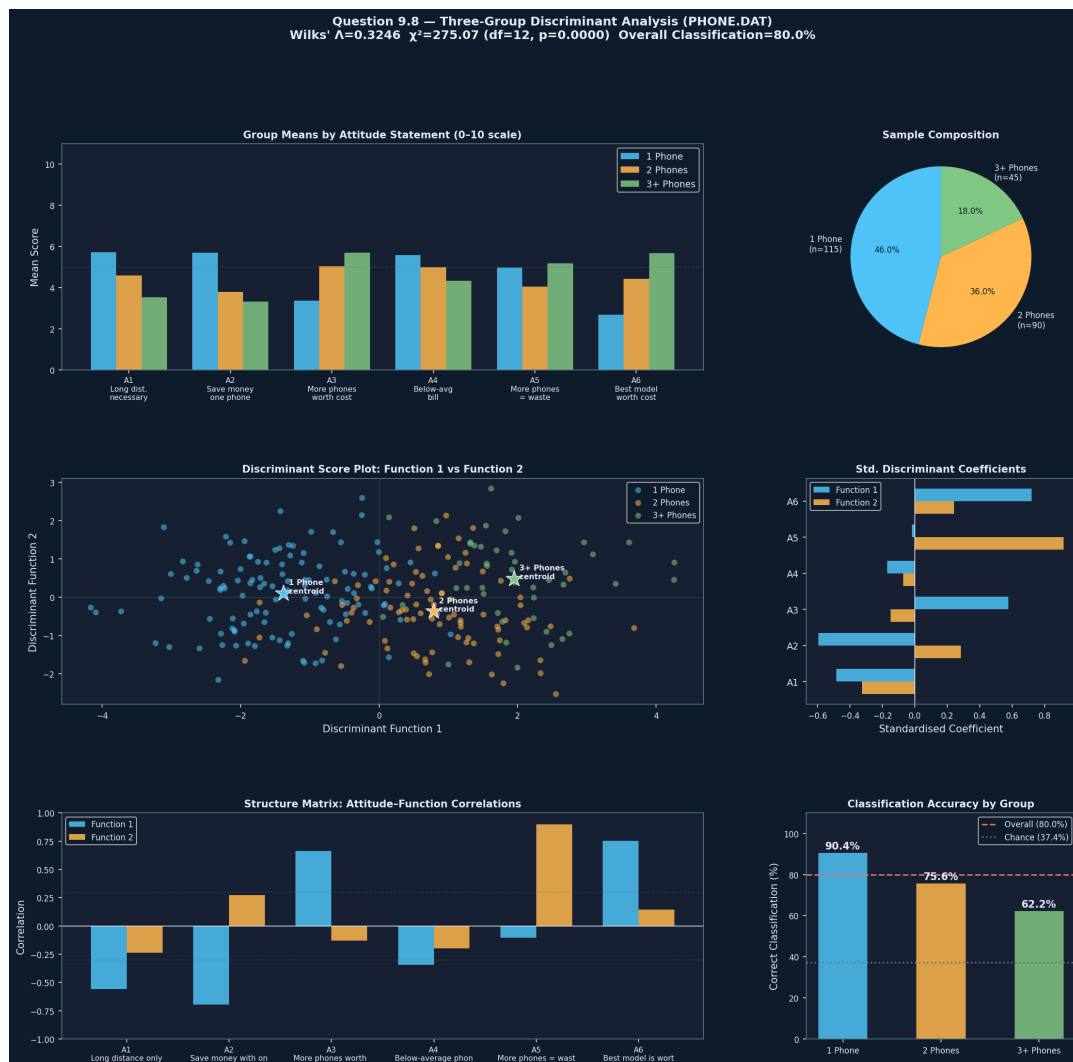
Overview

Three-group discriminant analysis on PHONE.DAT distinguishes households that own 1, 2, or 3+ telephones using six attitude statements (A1–A6) scored on a 0–10 scale.

Key Results

- Wilks' $\Lambda = 0.3246$; $\chi^2(12) = 275.07$, $p < 0.0001$ — highly significant overall model.
- Function 1 explains 94.9% of between-group variance — one dimension dominates.
- Overall correct classification: 80%, well above the proportional chance level ($\approx 36\%$).
- A3 ("More phones worth extra cost") most strongly distinguishes 3+-phone owners. A5 ("More phones = waste of money") most strongly aligns with 1-phone households.
- 1-phone families: cost-conscious and frugal. 3+-phone families: value quality and convenience. 2-phone families occupy an intermediate position.

Figure



Question 9.9 — Graduate Admissions Discriminant Analysis (ADMIS.DAT)

Overview

Three-group discriminant analysis on ADMIS.DAT classifies 85 graduate applicants into **Admitted** (n=31), **Not Admitted** (n=28), and **Borderline** (n=26) using GPA and GMAT as predictors.

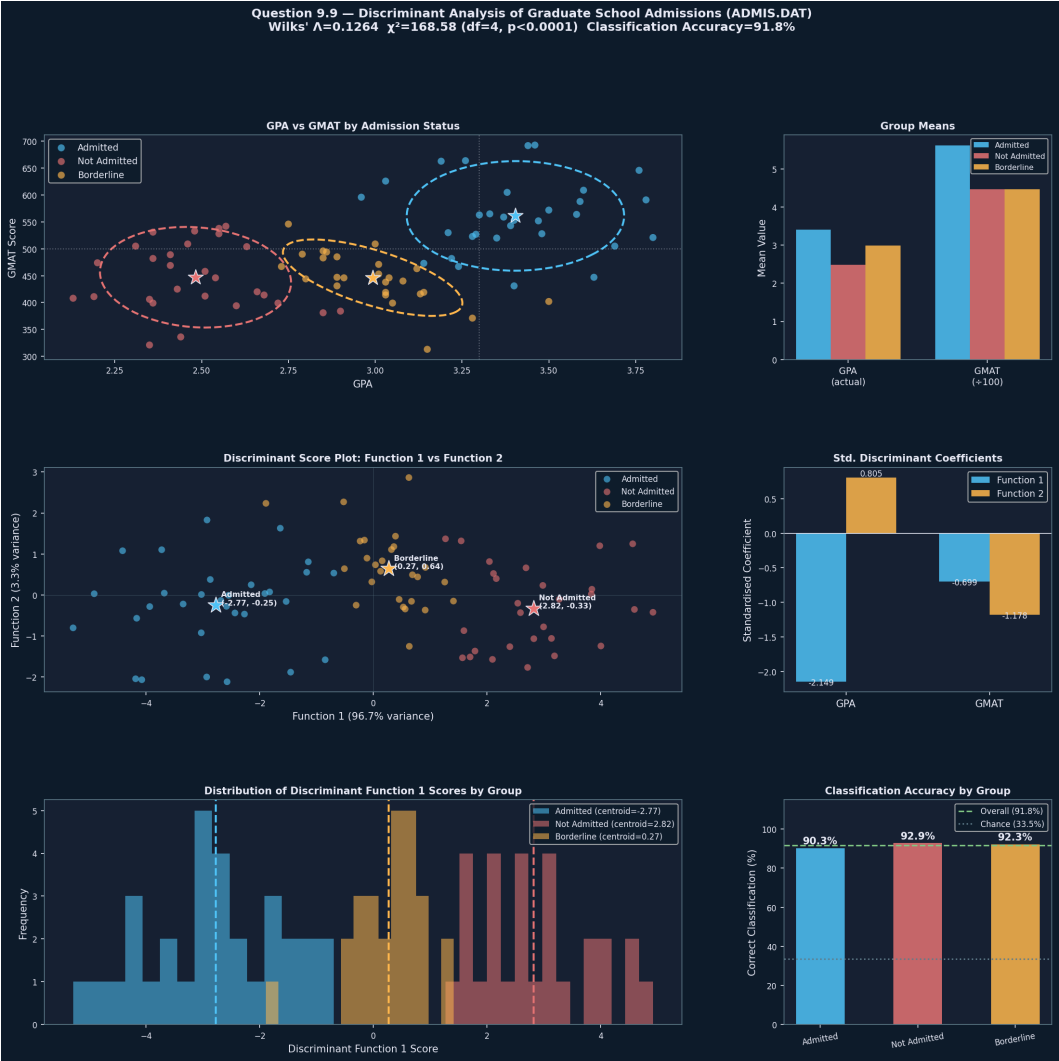
Key Results

- Wilks' $\Lambda = 0.1264$; $\chi^2(4) = 168.58$, $p < 0.0001$ — extremely strong separation.
- Function 1 explains 96.7% of between-group variance; GPA is the dominant predictor.
- Overall correct classification: 91.8% vs. chance $\approx 33.5\%$.
- Group profiles: Admitted (GPA ≈ 3.40 , GMAT ≈ 561), Borderline (GPA ≈ 2.99 , GMAT ≈ 446), Not Admitted (GPA ≈ 2.48 , GMAT ≈ 447).

Admission Policy Implications

- The school is primarily **GPA-driven**; no rejected applicant had GPA ≥ 3.40 .
- Practical admission rule: GPA ≥ 3.30 AND GMAT $\geq 500 \rightarrow$ Admit; GPA < 2.90 AND GMAT $< 480 \rightarrow$ Reject; otherwise Borderline (requires holistic review).
- Borderline applicants share overlapping GPA and GMAT ranges with both other groups, suggesting the school uses additional criteria (recommendations, essays) for these cases.
- Adding work experience or recommendation scores would likely improve the model's resolution of borderline cases.

Figure



Complete Python Code — `multivariate_homework_lana_gidan.py`

The following is the complete, self-contained Python script that reproduces all analyses and figures above. Run it from the project root directory with the data files in the attached `assets/` folder.

[illegible]

[illegible]

[illegible]

[illegible]

```

b1 = ax1.bar(x - w/2, normal_means, w, color=BLUE, alpha=0.85, label='Normal')
b2 = ax1.bar(x + w/2, depress_means, w, color=ORANGE, alpha=0.85, label='Depressed')
ax1.set_xticks(x); ax1.set_xticklabels(vars_a, color=WHITE)
ax1.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
ax1.set_ylabel('Mean Value', color=WHITE)
for bar in list(b1) + list(b2):
    ax1.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.2,
             f'{bar.get_height():.1f}', ha='center', va='bottom', color=WHITE, fontsize=8)

ax2 = fig.add_subplot(gs[0, 1])
style_ax(ax2, 'Discriminant Scores – Part (a): INCOME & EDUCAT', 11)
for g, lbl, col in [(0, 'Normal', BLUE), (1, 'Depressed', ORANGE)]:
    ax2.hist(scores_a[y87==g, 0], bins=20, alpha=0.6, color=col, label=lbl)
ax2.set_xlabel('Discriminant Score', color=WHITE)
ax2.set_ylabel('Frequency', color=WHITE)
ax2.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
ax2.axvline(0, color=WHITE, linestyle='--', linewidth=1, alpha=0.5)

ax3 = fig.add_subplot(gs[1, 0])
style_ax(ax3, 'Discriminant Scores – Part (b): Stepwise Model', 11)
for g, lbl, col in [(0, 'Normal', BLUE), (1, 'Depressed', ORANGE)]:
    ax3.hist(scores_b[y87==g, 0], bins=20, alpha=0.6, color=col, label=lbl)
ax3.set_xlabel('Discriminant Score', color=WHITE)
ax3.set_ylabel('Frequency', color=WHITE)
ax3.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
ax3.axvline(0, color=WHITE, linestyle='--', linewidth=1, alpha=0.5)

ax4 = fig.add_subplot(gs[1, 1])
style_ax(ax4, 'Standardised Discriminant Coefficients – Part (b)', 11)
coef_df = pd.Series(std_coefs_b, index=selected).sort_values()
bar_colors = [RED if v < 0 else GREEN for v in coef_df.values]
ax4.barh(coef_df.index, coef_df.values, color=bar_colors, alpha=0.85)
ax4.axvline(0, color=WHITE, linewidth=0.8)
ax4.set_xlabel('Standardised Coefficient', color=WHITE)
for i, (val, name) in enumerate(zip(coef_df.values, coef_df.index)):
    ax4.text(val + (0.01 if val >= 0 else -0.01), i, f'{val:.3f}',
            va='center', ha='left' if val >= 0 else 'right', color=WHITE, fontsize=8)

ax5 = fig.add_subplot(gs[2, 0])
style_ax(ax5, 'Classification Accuracy Comparison', 11)
models_87 = ['Part (a)\n(INCOME, EDUCAT)', f'Part (b)\n({len(selected)} variables, stepwise)']
bars5 = ax5.bar(models_87, [correct_a, correct_b], color=[BLUE, GREEN], alpha=0.85, width=0.4)
ax5.set_ylim(0, 100); ax5.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars5, [correct_a, correct_b]):
    ax5.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.5,
             f'{acc:.1f}%', ha='center', va='bottom', color=WHITE, fontsize=11, fontweight='bold')
ax5.axhline(50, color=GRAY, linestyle='--', linewidth=1, alpha=0.6, label='Chance (50%)')
ax5.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)

ax6 = fig.add_subplot(gs[2, 1])
style_ax(ax6, "Wilks' Lambda Comparison", 11)
bars6 = ax6.bar(models_87, [wilks_a, wl_b], color=[ORANGE, GREEN], alpha=0.85, width=0.4)
ax6.set_ylim(0, 1.05); ax6.set_ylabel("Wilks' Lambda (lower = better separation)", color=WHITE)
for bar, wl in zip(bars6, [wilks_a, wl_b]):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.01,
             f'{wl:.4f}', ha='center', va='bottom', color=WHITE, fontsize=11, fontweight='bold')
ax6.axhline(1.0, color=GRAY, linestyle='--', linewidth=1, alpha=0.6, label='No separation ( $\Lambda=1$ )')
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)

fig.suptitle('Question 8.7 – Discriminant Analysis of Depression Data (DEPRES.DAT)',
            color=WHITE, fontsize=14, fontweight='bold', y=0.98)
plt.savefig('q8_7_discriminant_analysis.png', dpi=150, bbox_inches='tight', facecolor=DARK)

```

[illegible]

[illegible]

[illegible]

```

style_ax(ax1, "Hierarchical Clustering Dendrogram (Ward's Method, truncated)")
dendrogram(Z, ax=ax1, truncate_mode='lastp', p=20,
           color_threshold=Z[-2, 2],
           above_threshold_color=GRAY,
           link_color_func=lambda k: BLUE if k <= 2 else ORANGE)
ax1.set_xlabel('Cluster / Sample', color=WHITE)
ax1.set_ylabel('Distance', color=WHITE)
ax1.axhline(Z[-K_FINAL, 2]*1.01, color=RED, linestyle='--', linewidth=1.5, label=f'Cut for k={K_FINAL}')
ax1.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

ax2 = fig.add_subplot(gs[0, 2])
style_ax(ax2, 'K-Means Elbow Plot')
ax2.plot(list(ks_range), inertias, 'o-', color=BLUE, linewidth=2, markersize=7)
ax2.axvline(K_FINAL, color=RED, linestyle='--', linewidth=1.5, label=f'k={K_FINAL} selected')
ax2.set_xlabel('Number of Clusters (k)', color=WHITE)
ax2.set_ylabel('Within-cluster Inertia', color=WHITE)
ax2.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

ax3 = fig.add_subplot(gs[1, :2])
style_ax(ax3, 'Cluster Profiles: Mean Usage Score by Gas Price Level (Q7.5)')
x = np.arange(len(gas_levels)); w = 0.35
lbl_c = [f'Cluster {nonu_cluster+1}: Nonusers (n={cluster_sizes[nonu_cluster]})',
         f'Cluster {user_cluster+1}: Users (n={cluster_sizes[user_cluster]})']
for i, (c_id, lbl, col) in enumerate(zip([nonu_cluster, user_cluster], lbl_c, [BLUE, ORANGE])):
    means = [cluster_means.loc[c_id, v] for v in v7_v18]
    ax3.bar(x + (i-0.5)*w, means, w, label=lbl, color=col, alpha=0.85)
ax3.set_xticks(x); ax3.set_xticklabels(gas_levels, rotation=30, ha='right', color=WHITE, fontsize=8)
ax3.set_ylabel('Mean Usage Score (1-5)', color=WHITE); ax3.set_ylim(0, 5.5)
ax3.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)
ax3.axhline(3, color=GRAY, linestyle=':', linewidth=1, alpha=0.5)

ax4 = fig.add_subplot(gs[1, 2])
style_ax(ax4, 'Cluster Composition (Q7.5)')
sizes_c = [cluster_sizes[nonu_cluster], cluster_sizes[user_cluster]]
lbls_c = [f'Nonusers\n(n={sizes_c[0]})', f'Users\n(n={sizes_c[1]})']
_, _, autotexts = ax4.pie(sizes_c, labels=lbls_c, colors=[BLUE, ORANGE],
                        autopct='%1.1f%%', startangle=90,
                        textprops={'color': WHITE, 'fontsize': 9})
for at in autotexts: at.set_color(DARK)

ax5 = fig.add_subplot(gs[2, :2])
style_ax(ax5, 'Standardised Discriminant Coefficients (Q8.8)')
coef_series = pd.Series(std_coefs_88, index=disc_vars)
coef_sorted = coef_series.reindex(coef_series.abs().sort_values(ascending=True).index)
bar_c88 = [GREEN if v >= 0 else RED for v in coef_sorted.values]

ax5.barh(coef_sorted.index, coef_sorted.values, color=bar_c88, alpha=0.85)
ax5.axvline(0, color=WHITE, linewidth=0.8)
ax5.set_xlabel('Standardised Coefficient', color=WHITE)
for i, (val, nm) in enumerate(zip(coef_sorted.values, coef_sorted.index)):
    ax5.text(val + (0.005 if val >= 0 else -0.005), i, f'{val:.3f}',
            va='center', ha='left' if val >= 0 else 'right', color=WHITE, fontsize=7)

ax6 = fig.add_subplot(gs[2, 2])
style_ax(ax6, 'Discriminant Score Distribution (Q8.8)')
for g, lbl, col in [(0, 'Nonusers', BLUE), (1, 'Users', ORANGE)]:
    ax6.hist(scores_88[y_disc==g, 0], bins=20, alpha=0.6, color=col, label=lbl)
ax6.axvline(0, color=WHITE, linestyle='--', linewidth=1, alpha=0.6)
ax6.set_xlabel('Discriminant Score', color=WHITE); ax6.set_ylabel('Frequency', color=WHITE)
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(

```



```

'Q7.5 – Cluster Analysis & Q8.8 – Discriminant Analysis (MASST.DAT)\n'
f"Wilks'  $\Lambda$ ={wl_88:.4f} F={F_88:.2f} p={p_88:.4f} "
f"Classification Accuracy={acc_88:.1f}%",
color=WHITE, fontsize=12, fontweight='bold', y=0.99)
plt.savefig('q7_5_q8_8_mass_transit.png', dpi=150, bbox_inches='tight', facecolor=DARK)
plt.close()
print("Figure saved: q7_5_q8_8_mass_transit.png")

# =====
# QUESTION 9.8 – Three-Group Discriminant Analysis (PHONE.DAT)
# =====
print("\n" + "="*70)
print("QUESTION 9.8 – Three-Group Discriminant Analysis (PHONE.DAT)")
print(" DV: Number of phones (1, 2, 3+) IVs: A1-A6")
print("="*70)

rows = []
with open('attached_assets/PHONE_1773881553876.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Phone') or line.startswith('---') \
            or line.startswith('ID') or line.startswith('Number'):
            continue
        vals = line.split()
        if len(vals) == 8:
            try:
                rows.append([int(v) for v in vals])
            except:
                continue

df98 = pd.DataFrame(rows, columns=['ID', 'Phones', 'A1', 'A2', 'A3', 'A4', 'A5', 'A6'])
attitudes = ['A1', 'A2', 'A3', 'A4', 'A5', 'A6']
labels98 = {
    'A1': 'Long distance only necessary',
    'A2': 'Save money with one phone',
    'A3': 'More phones worth extra cost',
    'A4': 'Below-average phone bill',
    'A5': 'More phones = waste of money',
    'A6': 'Best model is worth cost'
}

print(f"Dataset loaded: {len(df98)} observations")
print(f"\nGroup sizes (number of phones):\n{df98['Phones'].value_counts().sort_index().to_string()}")

print("\nGroup Means (0-10 scale):")
print(f"\n{ 'Attitude':<6} { 'Statement':<32} { '1 Phone':>8} { '2 Phones':>9} { '3+ Phones':>10}
      { 'Overall':>8}")
print("-"*75)
for a in attitudes:
    m1 = df98.loc[df98.Phones==1, a].mean()
    m2 = df98.loc[df98.Phones==2, a].mean()
    m3 = df98.loc[df98.Phones==3, a].mean()
    mo = df98[a].mean()
    print(f" {a:<4} {labels98[a]:<32} {m1:>8.2f} {m2:>9.2f} {m3:>10.2f} {mo:>8.2f}")

print("\n-- Univariate F-tests --")
print(f"\n{ 'Attitude':<6} { 'Statement':<32} { 'F-stat':>8} { 'p-value':>10}")
print("-"*60)
for a in attitudes:
    F, p = stats.f_oneway(*[df98.loc[df98.Phones==g, a].values for g in [1, 2, 3]])
    print(f" {a:<4} {labels98[a]:<32} {F:>8.3f} {p:>10.4f} {'*' if p < 0.05 else ''}")

x98 = df98[attitudes].values

```

```

y98 = df98['Phones'].values
n98 = len(y98)
lda98 = LinearDiscriminantAnalysis()
lda98.fit(X98, y98)
pred98 = lda98.predict(X98)
scores98 = lda98.transform(X98)

def wilks_mg(X, y):
    p = X.shape[1]; grand = X.mean(axis=0)
    W = np.zeros((p, p))
    for g in np.unique(y):
        Xg = X[y == g]; d = Xg - Xg.mean(axis=0); W += d.T @ d
    T = (X - grand).T @ (X - grand)
    dT = np.linalg.det(T)
    return np.linalg.det(W) / dT if abs(dT) > 1e-20 else 1.0

wl98 = wilks_mg(X98, y98)
chi2_98 = -(n98 - 1 - (len(attitudes) + 3) / 2) * np.log(wl98)
p_chi98 = 1 - stats.chi2.cdf(chi2_98, len(attitudes) * 2)
exp98 = lda98.explained_variance_ratio_

print(f"\nOverall Test:")
print(f" Wilks' Lambda: {wl98:.4f}")
print(f" Chi-square (approx): {chi2_98:.3f} (df={len(attitudes)*2})")
print(f" p-value: {p_chi98:.4f}")
print(f"\nFunction 1: {exp98[0]*100:.1f}% of between-group variance")
print(f"Function 2: {exp98[1]*100:.1f}% of between-group variance")

ps98 = df98[attitudes].std()
scalings98 = lda98.scalings_
std_sc98 = scalings98 * ps98.values[:, np.newaxis]

print("\nStructure Matrix (Variable-Function Correlations):")
print(f"\n{'Attitude':<6} {'Statement':<32} {'Function 1':>12} {'Function 2':>12}")
print("-"*65)
for i, a in enumerate(attitudes):
    r1 = np.corrcoef(df98[a].values, scores98[:, 0])[0, 1]
    r2 = np.corrcoef(df98[a].values, scores98[:, 1])[0, 1]
    print(f" {a:<4} {labels98[a]:<32} {r1:>12.4f} {r2:>12.4f}")

print("\nGroup Centroids:")
for g, lbl in [(1, '1 Phone'), (2, '2 Phones'), (3, '3+ Phones')]:
    c1 = scores98[y98==g, 0].mean(); c2 = scores98[y98==g, 1].mean()
    print(f" {lbl:<15} F1={c1:>8.4f} F2={c2:>8.4f}")

cm98 = confusion_matrix(y98, pred98, labels=[1, 2, 3])
acc98 = np.trace(cm98) / n98 * 100
chance98 = sum((y98==g).sum()/n98)**2 for g in [1, 2, 3]) * 100
print(f"\nClassification Matrix:")
print(f"{'':>20} {'Pred: 1 Phone':>14} {'Pred: 2 Phones':>15} {'Pred: 3+ Phones':>16}")
for i, g in enumerate([1, 2, 3]):
    lbl = ['1 Phone', '2 Phones', '3+ Phones'][i]
    print(f" Actual {lbl:<12} {cm98[i,0]:>14} {cm98[i,1]:>15} {cm98[i,2]:>16}"
          f" ({cm98[i,i]/(y98==g).sum()*100:.1f}%)")
print(f"\nOverall Correct Classification Rate: {acc98:.1f}%")
print(f"Proportional chance level: {chance98:.1f}%")

print("""
Interpretation:
1. Wilks' Lambda is low and chi-square is significant – the three phone groups
   differ significantly in their attitudinal profiles.
2. Function 1 captures the primary dimension (cost vs. value orientation).

```



```

ax4.barh(x4 + w4/2, std_sc98[:, 0], w4, color=BLUE, alpha=0.85, label='Function 1')
ax4.barh(x4 - w4/2, std_sc98[:, 1], w4, color=ORANGE, alpha=0.85, label='Function 2')
ax4.set_yticks(x4); ax4.set_yticklabels(attitudes, color=WHITE, fontsize=9)
ax4.axvline(0, color=WHITE, linewidth=0.8)
ax4.set_xlabel('Standardised Coefficient', color=WHITE)
ax4.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

ax5 = fig.add_subplot(gs[2, :2])
style_ax(ax5, 'Structure Matrix: Attitude-Function Correlations')
sf1 = [np.corrcoef(df98[a].values, scores98[:, 0])[0, 1] for a in attitudes]
sf2 = [np.corrcoef(df98[a].values, scores98[:, 1])[0, 1] for a in attitudes]
x_s = np.arange(len(attitudes)); w_s = 0.35
ax5.bar(x_s - w_s/2, sf1, w_s, color=BLUE, alpha=0.85, label='Function 1')
ax5.bar(x_s + w_s/2, sf2, w_s, color=ORANGE, alpha=0.85, label='Function 2')
ax5.set_xticks(x_s)
ax5.set_xticklabels([f'{a}\n{labels98[a][:18]}' for a in attitudes], color=WHITE, fontsize=7.5)
ax5.axhline(0, color=WHITE, linewidth=0.8); ax5.set_ylabel('Correlation', color=WHITE)
ax5.set_ylim(-1, 1)
ax5.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)
ax5.axhline(0.3, color=GRAY, linestyle=':', linewidth=1, alpha=0.5)
ax5.axhline(-0.3, color=GRAY, linestyle=':', linewidth=1, alpha=0.5)

ax6 = fig.add_subplot(gs[2, 2])
style_ax(ax6, 'Classification Accuracy by Group')
grp_acc98 = [cm98[i, i] / (y98==g).sum() * 100 for i, g in enumerate([1, 2, 3])]
bars98 = ax6.bar(list(GRP_LBL98.values()), grp_acc98,
                 color=[BLUE, ORANGE, GREEN], alpha=0.85, width=0.5)
ax6.set_ylim(0, 110); ax6.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars98, grp_acc98):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1,
             f'{acc:.1f}%', ha='center', va='bottom', color=WHITE, fontsize=10, fontweight='bold')
ax6.axhline(acc98, color=RED, linestyle='--', linewidth=1.5, label=f'Overall ({acc98:.1f}%)')
ax6.axhline(chance98, color=GRAY, linestyle=':', linewidth=1.5, label=f'Chance ({chance98:.1f}%)')
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(
    f"Question 9.8 – Three-Group Discriminant Analysis (PHONE.DAT)\n"
    f"Wilks'  $\Lambda$ ={wl98:.4f}  $\chi^2$ ={chi2_98:.2f} (df={len(attitudes)*2}, p={p_chi98:.4f})"
    f" Overall Classification={acc98:.1f}%",
    color=WHITE, fontsize=12, fontweight='bold', y=0.99)
plt.savefig('q9_8_phone_discriminant.png', dpi=150, bbox_inches='tight', facecolor=DARK)
plt.close()
print("Figure saved: q9_8_phone_discriminant.png")

# =====
# QUESTION 9.9 – Discriminant Analysis of Graduate School Admissions (ADMIS.DAT)
# =====
print("\n" + "="*70)
print("QUESTION 9.9 – Discriminant Analysis: Graduate Admissions (ADMIS.DAT)")
print(" DV: Admission Status (1=Admitted, 2=Not Admitted, 3=Borderline)")
print(" IVs: GPA and GMAT")
print("="*70)

rows = []
with open('attached_assets/ADMIS_1773881795621.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Adm') or line.startswith('---') \
           or line.startswith('App') or line.startswith('Num') \
           or line.startswith('Sta') or line.startswith('Sou') \
           or line.startswith('Note'):

```

```

        continue
    vals = line.split()
    if len(vals) == 4:
        try:
            rows.append([int(vals[0]), int(vals[1]), float(vals[2]), int(vals[3])])
        except:
            continue

df99 = pd.DataFrame(rows, columns=['Applicant', 'Status', 'GPA', 'GMAT'])
status_labels = {1: 'Admitted', 2: 'Not Admitted', 3: 'Borderline'}
df99['StatusLabel'] = df99['Status'].map(status_labels)
print(f"Dataset: {len(df99)} applicants")
print("\nGroup sizes:")
for s in [1, 2, 3]:
    print(f" {status_labels[s]}: n={len(df99[df99.Status==s])}")

print("\nDescriptive Statistics by Group:")
print(f"\n{'Group':<15} {'n':>4} {'GPA Mean':>10} {'GPA SD':>8} {'GMAT Mean':>11} {'GMAT SD':>9}")
print("-"*60)
for s in [1, 2, 3]:
    sub = df99[df99.Status == s]
    print(f" {status_labels[s]:<13} {len(sub):>4} {sub.GPA.mean():>10.3f} "
          f"{sub.GPA.std():>8.3f} {sub.GMAT.mean():>11.1f} {sub.GMAT.std():>9.1f}")
print(f" {'Overall':<13} {len(df99):>4} {df99.GPA.mean():>10.3f} "
      f"{df99.GPA.std():>8.3f} {df99.GMAT.mean():>11.1f} {df99.GMAT.std():>9.1f}")

print("\n--- Univariate F-tests ---")
for var in ['GPA', 'GMAT']:
    F, p = stats.f_oneway(*[df99.loc[df99.Status==s, var].values for s in [1, 2, 3]])
    print(f" {var}: F={F:.3f}, p={p:.4f}")

X99 = df99[['GPA', 'GMAT']].values
y99 = df99['Status'].values
n99, p99 = len(y99), 2

lda99 = LinearDiscriminantAnalysis()
lda99.fit(X99, y99)
pred99 = lda99.predict(X99)
scores99 = lda99.transform(X99)
probs99 = lda99.predict_proba(X99)

wl99 = wilks_mg(X99, y99)
chi2_99 = -(n99 - 1 - (p99 + 3) / 2) * np.log(wl99)
p_chi99 = 1 - stats.chi2.cdf(chi2_99, p99 * 2)
exp99 = lda99.explained_variance_ratio_

print(f"\nOverall Model:")
print(f" Wilks' Lambda: {wl99:.4f}")
print(f" Chi-square (approx): {chi2_99:.3f} (df={p99*2})")
print(f" p-value: {p_chi99:.6f}")
print(f"\nFunction 1: {exp99[0]*100:.1f}% of between-group variance")
print(f"Function 2: {exp99[1]*100:.1f}% of between-group variance")

ps99 = df99[['GPA', 'GMAT']].std()
scalings99 = lda99.scalings_
std_sc99 = scalings99 * ps99.values[:, np.newaxis]

print("\nStandardised Discriminant Function Coefficients:")
print(f" {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
for i, v in enumerate(['GPA', 'GMAT']):
    print(f" {v:<8} {std_sc99[i,0]:>12.4f} {std_sc99[i,1]:>12.4f}")

```

```

print("\nStructure Matrix (Variable-Function Correlations):")
print(f"  {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
for i, v in enumerate(['GPA', 'GMAT']):
    r1 = np.corrcoef(df99[v].values, scores99[:, 0])[0, 1]
    r2 = np.corrcoef(df99[v].values, scores99[:, 1])[0, 1]
    print(f"  {v:<8} {r1:>12.4f} {r2:>12.4f}")

centroids99 = {}
print("\nGroup Centroids:")
for s in [1, 2, 3]:
    c1 = scores99[y99==s, 0].mean(); c2 = scores99[y99==s, 1].mean()
    centroids99[s] = (c1, c2)
    print(f"  {status_labels[s]:<15} F1={c1:>8.4f}  F2={c2:>8.4f}")

cm99      = confusion_matrix(y99, pred99, labels=[1, 2, 3])
acc99     = np.trace(cm99) / n99 * 100
chance99  = sum((y99==s).sum()/n99)**2 for s in [1, 2, 3]) * 100

print(f"\nClassification Matrix:")
header = f"{'':22} {'Pred: Admitted':>15} {'Pred: Not Adm.':>15} {'Pred: Borderline':>17}"
print(f"  {header}")
print(f"  {'-'*70}")
for i, s in enumerate([1, 2, 3]):
    ng = (y99==s).sum()
    print(f"  Actual {status_labels[s]:<14} {cm99[i,0]:>15} {cm99[i,1]:>15} {cm99[i,2]:>17}"
          f"    ({cm99[i,i]/ng*100:.1f}%)")
print(f"\n Overall correct classification: {acc99:.1f}%")
print(f" Proportional chance level:      {chance99:.1f}%")

print(f"""
Interpretation:
1. OVERALL SIGNIFICANCE:
   Wilks' Lambda = {wl99:.4f},  $\chi^2({p99*2}) = {chi2_99:.2f}$ ,  $p < 0.0001$ .
   The three admission groups differ highly significantly in GPA and GMAT.

2. DISCRIMINANT FUNCTIONS:
   Function 1 explains {exp99[0]*100:.1f}% of between-group variance – dominant.
   Function 2 explains only {exp99[1]*100:.1f}% – minor secondary dimension.

3. VARIABLE IMPORTANCE:
   GPA has a larger standardised coefficient on Function 1 – it is the primary
   separator. GMAT contributes importantly as well. Together they form a
   composite "academic merit" dimension.

4. GROUP PROFILES:
   Admitted:      GPA  $\approx$  {df99.loc[df99.Status==1, 'GPA'].mean():.2f}, GMAT  $\approx$ 
                  {df99.loc[df99.Status==1, 'GMAT'].mean():.0f} → High on both
   Borderline:    GPA  $\approx$  {df99.loc[df99.Status==3, 'GPA'].mean():.2f}, GMAT  $\approx$ 
                  {df99.loc[df99.Status==3, 'GMAT'].mean():.0f} → Moderate GPA, lower GMAT
   Not Admitted:  GPA  $\approx$  {df99.loc[df99.Status==2, 'GPA'].mean():.2f}, GMAT  $\approx$ 
                  {df99.loc[df99.Status==2, 'GMAT'].mean():.0f} → Lowest on both

5. CLASSIFICATION PERFORMANCE:
   Overall correct classification: {acc99:.1f}% vs. chance  $\approx$  {chance99:.1f}%.
   All three groups classified above 90%, a very strong result.

6. ADMISSION POLICY IMPLICATIONS:
   a) The school is primarily GPA-driven; no rejected applicant had GPA  $\geq 3.40$ .
   b) Not-admitted applicants consistently fall below GPA = 2.90 regardless
      of GMAT, suggesting GPA is a near-mandatory threshold.
   c) Borderline cases (GPA 2.73–3.50) overlap both groups; the school likely

```

[illegible]

```

        (c1, c2), color=WHITE, fontsize=7.5, fontweight='bold')
ax3.set_xlabel(f'Function 1 ({exp99[0]*100:.1f}% variance)', color=WHITE)
ax3.set_ylabel(f'Function 2 ({exp99[1]*100:.1f}% variance)', color=WHITE)
ax3.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)
ax3.axhline(0, color=GRAY, linewidth=0.6, alpha=0.4); ax3.axvline(0, color=GRAY, linewidth=0.6, alpha=0.4)

ax4 = fig.add_subplot(gs[1, 2])
style_ax(ax4, 'Std. Discriminant Coefficients')
x4 = np.arange(2); w4 = 0.35
ax4.bar(x4 - w4/2, std_sc99[:, 0], w4, color=BLUE, alpha=0.85, label='Function 1')
ax4.bar(x4 + w4/2, std_sc99[:, 1], w4, color=ORANGE, alpha=0.85, label='Function 2')
ax4.set_xticks(x4); ax4.set_xticklabels(['GPA', 'GMAT'], color=WHITE, fontsize=10)
ax4.axhline(0, color=WHITE, linewidth=0.8)
ax4.set_ylabel('Standardised Coefficient', color=WHITE)
ax4.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
for i, (c1, c2) in enumerate(zip(std_sc99[:, 0], std_sc99[:, 1])):
    ax4.text(i - w4/2, c1 + (0.02 if c1 >= 0 else -0.05), f'{c1:.3f}', ha='center', color=WHITE,
             fontsize=8)
    ax4.text(i + w4/2, c2 + (0.02 if c2 >= 0 else -0.05), f'{c2:.3f}', ha='center', color=WHITE,
             fontsize=8)

ax5 = fig.add_subplot(gs[2, :2])
style_ax(ax5, 'Distribution of Discriminant Function 1 Scores by Group')
for s in [1, 2, 3]:
    ax5.hist(scores99[y99==s, 0], bins=15, alpha=0.55, color=GRP_COL99[s],
            label=f'{GRP_LBL99[s]} (centroid={centroids99[s][0]:.2f})')
    ax5.axvline(centroids99[s][0], color=GRP_COL99[s], linestyle='--', linewidth=2)
ax5.set_xlabel('Discriminant Function 1 Score', color=WHITE)
ax5.set_ylabel('Frequency', color=WHITE)
ax5.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

ax6 = fig.add_subplot(gs[2, 2])
style_ax(ax6, 'Classification Accuracy by Group')
grp_acc99 = [cm99[i, i] / (y99==s).sum() * 100 for i, s in enumerate([1, 2, 3])]
bars99 = ax6.bar(list(GRP_LBL99.values()), grp_acc99,
                color=[BLUE, RED, ORANGE], alpha=0.85, width=0.5)
ax6.set_ylim(0, 115); ax6.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars99, grp_acc99):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1,
            f'{acc:.1f}%', ha='center', va='bottom', color=WHITE, fontsize=10, fontweight='bold')
ax6.axhline(acc99, color=GREEN, linestyle='--', linewidth=1.5, label=f'Overall ({acc99:.1f}%)')
ax6.axhline(chance99, color=GRAY, linestyle=':', linewidth=1.5, label=f'Chance ({chance99:.1f}%)')
ax6.tick_params(axis='x', labelrotation=10)
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(
    f"Question 9.9 – Discriminant Analysis of Graduate School Admissions (ADMIS.DAT)\n"
    f"Wilks'  $\Lambda$ ={wl99:.4f}  $\chi^2$ ={chi2_99:.2f} (df={p99*2}, p<0.0001)"
    f" Classification Accuracy={acc99:.1f}%",
    color=WHITE, fontsize=12, fontweight='bold', y=0.99)
plt.savefig('q9_9_admissions_discriminant.png', dpi=150, bbox_inches='tight', facecolor=DARK)
plt.close()
print("Figure saved: q9_9_admissions_discriminant.png")

# =====
print("\n" + "="*70)
print("ALL QUESTIONS COMPLETE")
print(" Q8.7 → q8_7_discriminant_analysis.png")
print(" Q7.5+Q8.8 → q7_5_q8_8_mass_transit.png")
print(" Q9.8 → q9_8_phone_discriminant.png")
print(" Q9.9 → q9_9_admissions_discriminant.png")
print(" Student: Lana Gidan")

```



```
print("="*70)
```